

AI-Powered Suspicious Activity Detection

An Autonomous, Query-Aware Agentic System for AML Compliance

Implementation Plan & Delivery Roadmap

From Solution Design to a Working, Demo-Ready Agentic System

Scope of this document

Phased build plan, sprint-wise task breakdown, module-level implementation detail, team roles, testing & validation strategy, deployment plan, risk register, and milestone-based delivery schedule for the AML Suspicious Activity Detection Agent described in the Solution Design document.

VIT Campus Hackathon — Problem Statement Response

Companion document to: Solution Design & Technical Architecture

July 24, 2026

Contents

1	Introduction & Purpose of This Document	2
1.1	Objectives of the Implementation Plan	2
1.2	Guiding Engineering Principles	2
2	Workstream Breakdown	2
3	Phased Implementation Plan	3
4	Sprint-Wise Schedule & Task Ownership	4
5	Visual Timeline	6
6	Module-Level Implementation Detail	6
6.1	Data Loader & Preprocessing Tool	6
6.2	Feature Engineering Tool	7
6.3	Anomaly / Suspicious-Pattern Detection Tool	7
6.4	Risk Classification Tool	7
6.5	Explanation Component	7
6.6	Escalation Policy Layer	8
6.7	Orchestrator Agent (Intent Parser, Planner, Controller)	8
6.8	API & Front-End	8
7	Testing & Validation Strategy	8
7.1	Test Data Strategy	9
8	Deployment & Demo Environment Plan	9
9	Team Roles & Responsibilities	10
10	Implementation Risk Register	10
11	Milestone & Deliverable Summary	11
12	Traceability: Implementation Plan to Problem Statement	12
13	Conclusion	12

1 Introduction & Purpose of This Document

The Solution Design document establishes *what* is being built: an LLM-driven orchestrator agent that dynamically plans and selectively invokes AML analysis tools (EDA, Feature Engineering, Anomaly Detection, Risk Classification, Explanation) rather than running a fixed sequential pipeline.

This Implementation Plan answers the complementary question: *how, in what order, by whom, and by when* the system gets built, tested, and demoed. It translates the architecture into an actionable, time-boxed engineering plan suitable for a hackathon delivery cycle, while remaining detailed enough to double as an engineering execution plan if extended into a production initiative.

1.1 Objectives of the Implementation Plan

- Break the system into independently buildable, independently testable components.
- Sequence work so that a **demoable, end-to-end path** exists as early as possible (walking skeleton first, sophistication later).
- Define clear **Definition of Done** criteria per module, tied back to the problem statement's success criteria.
- Provide a **sprint-wise schedule** with owners, dependencies, and effort estimates.
- Define the **testing, validation, and demo rehearsal** strategy so judge-facing behaviour (plan JSON, skipped-tool reasoning, explanations) is verified, not assumed.
- Capture **risks** specific to implementation (not just design) and how the team mitigates them under time pressure.

1.2 Guiding Engineering Principles

1. **Walking skeleton first.** Get a query → plan → dummy tool → report round-trip working on Day 1, before any tool is "smart."
2. **Tools before intelligence.** Build every tool as a pure, independently testable Python function first; wire the LLM planner in afterward.
3. **Fixtures over live data early.** Use small, hand-crafted synthetic CSVs with known injected structuring/smurfing cases so correctness can be verified by inspection before scaling to the full Kaggle datasets.
4. **Contract-first.** Freeze the `QuerySpec`, `ExecutionPlan`, and `ExecutionReport` JSON schemas (Section 4.1–4.2 and Section 8 of the Solution Design) on Day 1 so front-end, orchestrator, and tool work can proceed in parallel without integration surprises.
5. **Everything the judge sees must be inspectable.** Every "skip" decision, every score, every explanation must be traceable to a logged tool call — this is validated as an explicit test category, not left to chance.

2 Workstream Breakdown

The build is organized into six parallelizable workstreams. Table 1 maps each workstream to the corresponding Solution Design section and the primary skill required.

WS #	Workstream	Design Ref.	Primary Skill
WS-1	Data Layer & Schema Normalization (Data Loader Tool)	§6, §5.1	Data Engineering
WS-2	Feature Engineering Tool (pattern-specific feature families)	§5.3	ML / Pandas
WS-3	Anomaly Detection Tool (rule / statistical / ML / hybrid)	§5.4	ML
WS-4	Risk Classification + Escalation Policy Layer	§5.5, §5.7	Backend / Business Logic
WS-5	Orchestrator Agent (Intent Parser, Planner, Controller, Explainer)	§4, §5.6	LLM / Agent Eng.
WS-6	API, Front-End & Report Rendering	§7, §8	Full-stack

Table 1: Workstream to Solution-Design mapping

Dependency rule of thumb: WS-1 blocks WS-2/WS-3; WS-2 & WS-3 block WS-4; WS-5 depends on stubbed versions of WS-1–WS-4 being callable (even as mocks) from Day 1, and on real implementations by integration day; WS-6 can be built against the frozen `ExecutionReport` schema in parallel with everything else.

3 Phased Implementation Plan

The build is split into four phases. Each phase ends with a concrete, demonstrable checkpoint rather than an abstract "progress" milestone.

Phase 0: Foundations & Contracts

Day 1

- Freeze `QuerySpec`, `ExecutionPlan`, and `ExecutionReport` JSON schemas.
- Stand up repo structure, tool registry interface (`tool_name` → `callable` contract), and the `AgentController` skeleton with *stub* tools that return hard-coded JSON.
- Generate/curate the synthetic demo dataset (Section 6.2 of Solution Design) with injected structuring, smurfing, and one clean high-volume "control" customer.
- **Checkpoint:** A hard-coded query returns a hard-coded `ExecutionReport` end-to-end through the real controller loop (no real tool logic yet, no real LLM calls yet).

Phase 1: Core Tools (Deterministic Half)

Days 2–3

- Implement Data Loader & Preprocessing Tool with column-mapping/normalization layer.
- Implement Feature Engineering Tool for the three feature families needed by the reference queries: near-threshold count (structuring), velocity, amount-deviation z-score.
- Implement Anomaly Detection Tool: rule engine path first (exact-threshold logic), then statistical z-score/IQR path, then IsolationForest/LOF path.
- Implement Risk Classification Tool with percentile-based cohort thresholds blended with the hard business rules from §5.5 of the Solution Design.
- Implement the deterministic Escalation Policy Layer (risk band → action table).
- **Checkpoint:** Every tool passes unit tests in isolation, using the synthetic dataset, with no LLM involved at all — i.e., the system's "numbers come from code" guarantee is testable independently of the agent.

Phase 2: Agentic Layer (LLM Half)**Days 3–4**

- Implement Intent & Entity Extraction as a constrained structured-output LLM call producing a valid `QuerySpec`.
- Implement the Dynamic Execution Planner: LLM call constrained to the registered tool list, producing an `ExecutionPlan` with `reasoning` and `skipped_tools`.
- Add the plan-validation layer: reject/repair plans referencing unregistered tools or missing required arguments (Risks table, Solution Design §11).
- Wire the `AgentController` tool-execution loop to call *real* Phase-1 tools using the validated plan.
- Implement the Explanation Component as a constrained LLM call that is only given already-computed numeric facts (never allowed to invent figures).
- **Checkpoint:** All three reference queries (§4.4 of Solution Design) produce distinct, correct execution paths end-to-end, with visibly different tools invoked/skipped per query.

Phase 3: Reporting, Hardening & Demo Readiness**Day 5**

- Build the final `ExecutionReport` assembler (Section 8 schema) including charts (amount distribution, flagged timeline).
- Build the FastAPI `/query` endpoint and the Streamlit/React front-end for judges to type a query and see the live plan + report.
- Add PII hashing/pseudonymization for customer identifiers in all logs and displayed output.
- Add caching for repeated `QuerySpec`→`Plan` shapes to reduce latency.
- Full regression pass across the four demo queries (broad, pattern-targeted, rule-based, entity-lookup) plus 2–3 adversarial/edge queries.
- Dry-run the Demo Walkthrough Plan (Solution Design §9) at least twice, timed.
- **Checkpoint:** System is demo-ready; every judge-facing claim (plan reasoning, skipped-tool justification, explanation, escalation) is verified against logged tool output, not just eyeballed.

4 Sprint-Wise Schedule & Task Ownership

Table 3 gives a concrete day-by-day breakdown assuming a 5-day hackathon build window with a 4–6 person team. Effort is expressed in person-hours; owners are role labels (map to actual team members).

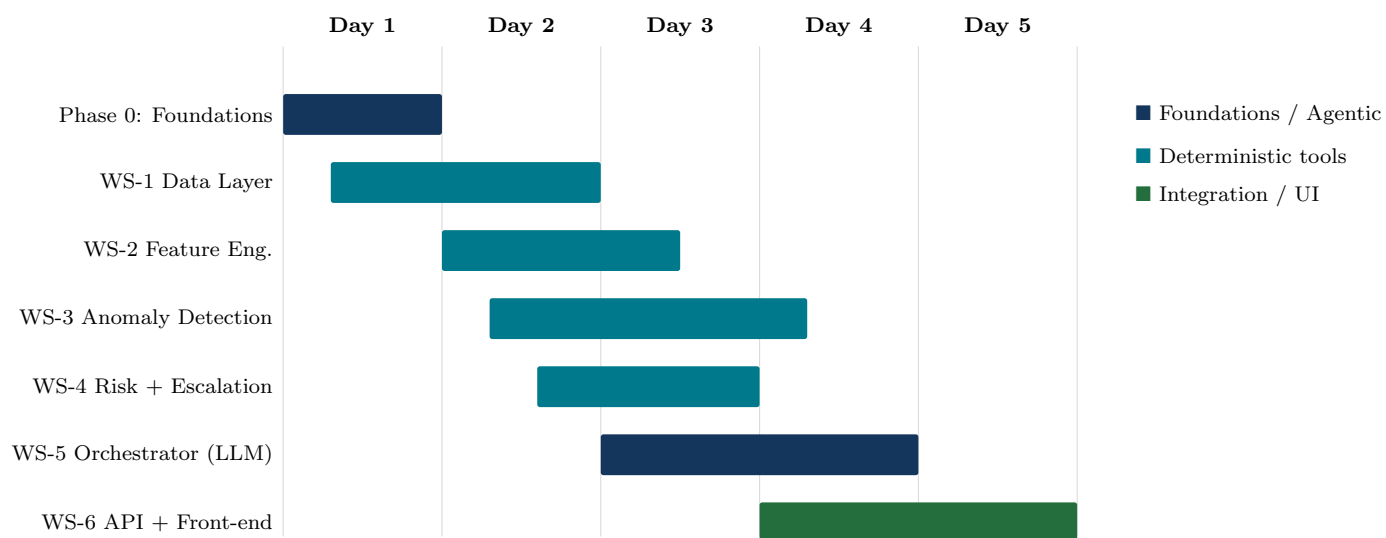
Day	Task	Owner (role)	Effort (hrs)	Depends on
Day 1	Freeze <code>QuerySpec</code> / <code>ExecutionPlan</code> / <code>ExecutionReport</code> schemas	Tech Lead	2	—
Day 1	Repo scaffold, tool registry interface, <code>AgentController</code> skeleton	Backend Eng.	3	Schemas

Day	Task	Owner (role)	Effort (hrs)	Depends on
Day 1	Curate synthetic dataset with injected structuring/smurfing cases	Data Eng.	4	—
Day 1	Stub all 5 tools returning hard-coded valid JSON	Backend Eng.	2	Registry
Day 1	Walking-skeleton checkpoint demo (internal)	Whole team	1	Above
Day 2	Data Loader + column-mapping/normalization layer	Data Eng.	5	Dataset
Day 2	Feature Engineering: near-threshold, velocity features	ML Eng.	6	Data Loader
Day 2	Rule Engine path of Anomaly Detection Tool	ML Eng.	3	Feature Eng.
Day 2	Risk Classification: percentile thresholds + business rules	Backend Eng.	4	—
Day 3	Statistical (z-score/IQR) Anomaly Detection path	ML Eng.	4	Feature Eng.
Day 3	ML path: IsolationForest / LOF + hybrid combiner	ML Eng.	6	Statistical path
Day 3	Escalation Policy Layer (deterministic table)	Backend Eng.	2	Risk Classification
Day 3	Unit tests for all Phase-1 tools on synthetic data	QA / Backend	4	All tools
Day 3	Intent & Entity Extraction LLM call (QuerySpec)	Agent Eng.	5	Schemas
Day 4	Dynamic Execution Planner LLM call + few-shot examples	Agent Eng.	6	QuerySpec
Day 4	Plan-validation / repair layer	Agent Eng.	3	Planner
Day 4	Wire AgentController to real tools via validated plan	Backend Eng.	4	Planner, Tools
Day 4	Explanation Component (constrained, number-grounded)	Agent Eng.	4	Risk Classification
Day 4	End-to-end test: 3 reference queries produce distinct plans	QA	3	All above
Day 5	ExecutionReport assembler + charts	Backend Eng.	4	Full pipeline
Day 5	FastAPI /query endpoint	Backend Eng.	3	Report assembler

Day	Task	Owner (role)	Effort (hrs)	Depends on
Day 5	Front-end (Streamlit/React) for judges	Frontend Eng.	6	API
Day 5	PII hashing pass & caching layer	Backend Eng.	2	—
Day 5	Full regression + adversarial queries	QA	3	Everything
Day 5	Demo dry-run (x2, timed)	Whole team	2	Everything

Table 3: Day-wise task schedule with ownership and dependencies

5 Visual Timeline



6 Module-Level Implementation Detail

Each module below lists: purpose recap, concrete implementation steps, key libraries, input-s/outputs, and its **Definition of Done (DoD)**.

6.1 Data Loader & Preprocessing Tool

Implementation steps:

1. Write a column-mapping config (YAML/JSON) that maps arbitrary source column names to the internal schema (Solution Design §6.1).
2. Implement type coercion (timestamps to UTC, amounts to `float64`, currency normalization to a base currency using a static/lookup FX table for the demo).
3. Implement filter application (date range, `customer_id`, segment, country, `transaction_type`) directly at load time so downstream tools only ever see the minimal relevant slice.
4. Emit a `preprocessing_log` (rows dropped, nulls imputed, dedup count) for auditability.

Libraries: `pandas`, `duckdb` (for larger-than-memory filtering).

DoD: Given the same raw CSV, three different `QuerySpec.filters` produce three correctly-scoped DataFrames verified against expected row counts in unit tests.

6.2 Feature Engineering Tool

Implementation steps:

1. Implement each feature family as an independent function keyed by name, so the planner can request `feature_set="structuring"` etc. without computing unrelated features.
2. Structuring/smurfing: rolling-window (24h/7d/30d) count of near-threshold transactions and their ratio to total transactions per customer.
3. Velocity: per-customer transaction frequency per time bucket and inter-transaction time deltas.
4. Layering: counterparty network hop-count and fan-in/fan-out ratios (graph built with `networkx` on counterparty edges).
5. Amount deviation: per-customer and per-segment z-scores of transaction amount.

Libraries: `pandas`, `numpy`, `networkx`.

DoD: Each feature family has a unit test with a hand-crafted mini-DataFrame where the expected feature values are computed by hand and asserted exactly.

6.3 Anomaly / Suspicious-Pattern Detection Tool

Implementation steps:

1. Rule engine: implement as a small DSL/evaluator that takes the `explicit_rule.condition` string (or a parsed structured version of it) and applies it directly to the DataFrame.
2. Statistical path: z-score/IQR flagging on engineered features, returned with the exact feature and threshold that triggered the flag (for the Explanation Component to consume later).
3. ML path: `IsolationForest` and `LocalOutlierFactor` on the pattern-specific feature set; extract top contributing features via path-length attribution (Isolation Forest) or per-feature z-scores as a lightweight explainability proxy.
4. Hybrid path: run the rule engine first as a cheap pre-filter, then run ML scoring only on the surviving subset.

Libraries: `scikit-learn`, `PyOD`, `scipy.stats`.

DoD: On the synthetic dataset with injected structuring cases, the tool recovers the injected cases with a documented true-positive rate, and does *not* flag the clean high-volume control customer.

6.4 Risk Classification Tool

Implementation steps:

1. Compute percentile-based thresholds *within the filtered cohort* returned by the upstream tool call (never against a global fixed cutoff).
2. Layer in hard business rules: any exact rule-engine match $\Rightarrow$ minimum Medium; any prior-SAR flag on the entity (from `customers.csv.kyc.flags`) $\Rightarrow$ minimum High.
3. Return a risk band, the underlying continuous score, and the specific rule/threshold that determined the final band (for auditability).

DoD: Band assignment is deterministic and reproducible given the same score distribution; business-rule overrides are unit-tested independently of the percentile logic.

6.5 Explanation Component

Implementation steps:

1. Design the constrained prompt template that receives *only* the already-computed numeric facts (score, band, top contributing features, matched pattern) plus the original user query text.
2. Explicitly instruct the model to restate, never invent, numeric values.

3. Add a post-generation regex/number-match validation step that cross-checks every number in the generated explanation against the numbers passed into the prompt; regenerate or flag on mismatch.

DoD: 100% of numbers appearing in generated explanations are traceable to the input payload across a 20-case validation batch.

6.6 Escalation Policy Layer

Implementation steps:

1. Implement as a pure lookup table (risk band $\rightarrow$ action), with no LLM involvement at all, per Solution Design §5.7.
2. Attach the human-readable rationale string alongside the action for reviewer display.

DoD: Table-driven; adding a new band/action requires a one-line config change, verified by test.

6.7 Orchestrator Agent (Intent Parser, Planner, Controller)

Implementation steps:

1. Implement Intent & Entity Extraction as a structured-output (JSON-schema-constrained) call producing `QuerySpec`.
2. Implement the Planner as a second structured-output call, constrained to the registered tool list, that must additionally produce a natural-language **reasoning** string and an explicit `skipped_tools` list with a reason per skipped tool.
3. Seed the planner prompt with few-shot examples covering (at minimum) the three reference queries from the problem statement, plus 3–5 additional edge-case queries (ambiguous intent, multiple patterns in one query, no filters at all).
4. Implement plan validation: reject plans referencing unregistered tools or missing required arguments; on rejection, either re-prompt once with the validation error appended, or fall back to a safe default plan (full EDA + all tools) and flag the fallback in the report.
5. Implement the `AgentController` loop exactly as in Solution Design Listing 3, passing `context` forward between steps.

DoD: All three reference queries plus at least 3 novel queries not seen during prompt design produce valid, schema-conformant plans with correct tool selection (verified by a human-reviewed rubric, since this is inherently non-deterministic LLM output).

6.8 API & Front-End

Implementation steps:

1. FastAPI service exposing `POST /query` accepting `{"query": str}` and returning the full `ExecutionReport` JSON.
2. Front-end (Streamlit for speed, or React if a richer UI is wanted) that: (a) accepts free-text query input, (b) renders the live plan (steps + skipped tools + reasoning) as it is decided, (c) renders flagged entities in a sortable table with risk-band color coding, (d) renders explanations inline per flagged entity, (e) renders the summary metrics and charts.
3. Add a raw-JSON toggle so judges can inspect the exact `ExecutionReport` payload directly.

DoD: A judge can type any of the four demo queries into the running front-end and see the plan, report, and raw JSON without needing terminal access.

7 Testing & Validation Strategy

Test Level	What it Verifies	Method
Unit tests	Each tool function’s numeric correctness in isolation	<code>pytest</code> with hand-computed expected values on small fixtures
Contract tests	Every tool call and LLM output conforms to the frozen JSON schemas	JSON-schema validation (<code>pydantic</code> or <code>jjsonschema</code>) run on every tool/LLM I/O
Integration tests	The <code>AgentController</code> correctly sequences real tools per a given plan	End-to-end run on the synthetic dataset with known expected flagged entities
Agentic behaviour tests	The planner selects <i>different</i> tool subsets for different query types, matching Solution Design §4.4	Scripted assertions on <code>ExecutionPlan.steps</code> and <code>skipped_tools</code> for the 3 reference queries + edge cases
Explainability audit	No hallucinated numbers appear in generated explanations	Automated number cross-check against the input payload (Section 6.5 above)
Detection accuracy	True-positive rate on injected structuring/smurfing cases; false-positive rate on the clean control customer	Confusion-matrix style evaluation against the synthetic ground truth
Latency & load	Response time for typical queries stays within demo-acceptable bounds	Manual timing + simple load test with repeated queries to validate the <code>QuerySpec</code> → <code>Plan</code> cache
PII / privacy check	Customer identifiers are hashed in logs and on-screen output	Manual + <code>grep</code> -based check of log output and rendered report for raw identifiers

Table 4: Testing pyramid mapped to system layers

7.1 Test Data Strategy

- **Tier 1 — hand-crafted micro-fixtures** (10–20 rows): used for unit tests where exact expected outputs can be computed by hand.
- **Tier 2 — synthetic injected dataset** (few thousand rows, built from the custom generator in Solution Design §6.2): used for integration tests and the live demo, with known structuring/smurfing cases and one clean control customer.
- **Tier 3 — public reference datasets** (IBM AML synthetic dataset, PaySim): used as a stretch validation to demonstrate the system generalizes beyond the hand-built fixtures, time permitting.

8 Deployment & Demo Environment Plan

1. **Local-first development:** all modules runnable via a single `make dev` / `docker-compose up` command, with the synthetic dataset bundled in-repo so no external data dependency exists during judging.
2. **Environment variables:** LLM API key(s) injected via `.env`, never committed; fallback stub-mode flag so the system can run key-less for offline rehearsal.
3. **Persistence:** SQLite/DuckDB file for demo persistence of flags and prior-SAR-equivalent decisions, seeded with the synthetic `kyc.flags` data.
4. **Demo machine checklist:** confirm network access to the LLM API from the venue, pre-warm the LLM connection before judges arrive, have a recorded fallback video of the four demo queries as a contingency.

5. **Reset script:** one command that resets the demo database/logs between judge sessions so each run starts clean.

9 Team Roles & Responsibilities

Role	Primary Responsibility	Owns
Tech Lead	Schema freeze, architecture decisions, integration sign-off	QuerySpec / Plan / Report contracts
Data Engineer	Dataset curation, Data Loader Tool, schema normalization	<code>transactions.csv</code> , <code>customers.csv</code> , Data Loader Tool
ML Engineer	Feature Engineering + Anomaly Detection tools	Rule/statistical/ML/hybrid detection logic
Backend Engineer	AgentController, Risk Classification, Escalation Layer, API	Orchestration plumbing, FastAPI service
Agent/LLM Engineer	Intent extraction, planner prompt design, plan validation, explanation prompts	All LLM-facing prompt engineering
Frontend Engineer	Judge-facing UI, live plan rendering, chart rendering	Streamlit/React app
QA / Everyone	Test-writing, demo rehearsal, edge-case query design	Test suite, demo script

Table 5: Role-based ownership map

10 Implementation Risk Register

This extends the design-level risk table (Solution Design §11) with risks specific to *building* the system under a hackathon timeline.

Risk	Severity	Mitigation
LLM planner produces inconsistent plans across runs (non-determinism)	Medium	Set low temperature for planning calls; add plan-validation + safe-default fallback; lock a small, curated few-shot set covering all reference queries.
Integration between WS-5 (agent) and WS-1–WS-4 (tools) slips because contracts weren't frozen early	High	Freeze schemas on Day 1 (Phase 0) before any tool logic is written; build stub tools immediately so integration is validated continuously, not only at the end.

Risk	Severity	Mitigation
Time spent over-engineering the ML anomaly detector at the expense of the agentic/planning layer (the actual rubric differentiator)	High	Timebox WS-3 hybrid/ML path; rule + statistical paths are sufficient for a correct demo — treat IsolationForest/LOF as an enhancement, not a blocker.
Front-end work starts too late and becomes the critical-path bottleneck	Medium	Build the front-end against the frozen ExecutionReport schema and mocked data starting Day 2, in parallel with backend work, not after integration.
LLM API latency/availability issues during live judging	Medium	Cache QuerySpec→Plan mappings for the exact reference queries; keep a recorded fallback demo video; pre-warm connections before the session.
Synthetic dataset doesn't actually exercise all three reference-query behaviours distinctly	Medium	Design the synthetic generator <i>from</i> the three reference queries backward — i.e., explicitly construct cases that require different tool subsets — rather than generating generic random data first.
Team members block on each other due to shared files (e.g., everyone editing <code>controller.py</code>)	Low	Enforce the tool-registry pattern strictly: each tool lives in its own module with a single registration call, so WS-1 through WS-4 never touch the same file.

11 Milestone & Deliverable Summary

Milestone	When	Deliverable
M0	End of Day 1	Walking skeleton: query → stub plan → stub tools → report, running end-to-end
M1	End of Day 3	All deterministic tools (Data Loader, Feature Engineering, Anomaly Detection, Risk Classification, Escalation) implemented and unit-tested against synthetic data
M2	End of Day 4	Full agentic loop working: real LLM-driven QuerySpec + Execution-Plan + validated execution across all three reference queries, producing distinct tool-invocation paths
M3	End of Day 5	Demo-ready system: FastAPI + front-end, PII-safe, cached, regression-tested, demo-rehearsed against the Demo Walkthrough Plan (Solution Design §9)

Table 7: Milestone checkpoints with associated deliverables

12 Traceability: Implementation Plan to Problem Statement

Problem Statement Requirement	Where it is Implemented
Agent must not follow a fixed sequential pipeline; must dynamically construct an execution plan	Phase 2 (Orchestrator Agent), Section 6.7, WS-5
Extract intent, filters, and target AML pattern from natural language	Intent & Entity Extraction (Section 6.7, item 1)
Decide which tools to call, in what order, on which subset	Dynamic Execution Planner + plan validation (Section 6.7, items 2–4)
Load dataset, apply only relevant preprocessing	Data Loader & Preprocessing Tool (Section 6.1)
Run EDA selectively; skip for targeted/entity queries	Planner's <code>skipped_tools</code> logic; EDA Tool built as an on-demand module invoked only when <code>requires_full_eda=true</code>
Create AML features on demand (frequency, rolling sums, deviation, velocity, cash-out)	Feature Engineering Tool (Section 6.2)
Run anomaly/pattern detection via ML, statistical, or rule-based methods	Anomaly Detection Tool (Section 6.3)
Classify low/medium/high risk with context-appropriate thresholds	Risk Classification Tool (Section 6.4)
Generate human-readable, query-tied explanation per flag	Explanation Component (Section 6.5)
Recommend monitor / review / report action	Escalation Policy Layer (Section 6.6)
Return judge-inspectable structured output with decisions and rationale	ExecutionReport assembler + front-end raw-JSON toggle (Section 6.8)

13 Conclusion

This implementation plan converts the agentic architecture in the Solution Design document into a concrete, day-by-day build sequence with frozen contracts, parallelizable workstreams, and explicit Definition-of-Done criteria per module. By building the deterministic tools and the agentic/LLM layer as independently testable tracks — and by validating the three reference queries as an explicit, scripted test category rather than an informal demo check — the team can be confident that what is shown to judges on Day 5 reflects the actual, inspectable behaviour of the system, not a rehearsed happy path.

The milestone structure (M0–M3) ensures a demoable artifact exists from Day 1 onward, so risk is front-loaded into integration rather than discovered on the final day.